

MACHINE LEARNING FUNDAMENTALS SERIES

Attention Mechanisms in Seq2Seq Neural Networks

Direct Context Access, Similarity Scoring, SoftMax Normalization & Weighted Aggregation

1. Introduction and Problem Formulation

In standard Sequence-to-Sequence (Seq2Seq) Encoder-Decoder architectures, an input sentence of variable length is processed by Recurrent Neural Networks (such as LSTMs) to compress all input information into a single fixed-length **Context Vector**. While this approach performs adequately for short phrases (e.g., translating *"let's go"*), it encounters a severe information bottleneck when scaling to long sentences or large vocabularies.

The Long-Sentence Bottleneck Example

Consider the sentence: *"Don't eat the delicious looking and smelling pizza."*

In standard Seq2Seq models, unrolling LSTMs across many steps causes early tokens to be forgotten.

If the initial word **"Don't"** is lost during context vector compression, the translated output flips to its exact opposite: *"Eat the delicious looking and smelling pizza."*

Even though LSTMs provide separate memory paths for long and short-term states, forcing all historical information through a single bottleneck vector inevitably loses critical early tokens. The **Attention Mechanism** resolves this by creating **direct connections from every input step in the Encoder to every output step in the Decoder**, enabling the Decoder to dynamically "attend" to relevant input words.

2. System Architecture and Functional Blocks

An Attention-augmented Seq2Seq system consists of six primary functional components:

- Encoder Hidden State Storage:** Unlike standard Seq2Seq models that discard intermediate states, the Attention Encoder retains the hidden state vector $\mathbf{h}_i^{\text{enc}}$ produced at every input time step i .
- Decoder Hidden State Generation:** At step j , the Decoder LSTM processes the previous output token and memory states to produce a target hidden state $\mathbf{s}_j^{\text{dec}}$.
- Similarity Scoring Module:** Computes pairwise alignment scores between the current decoder state $\mathbf{s}_j^{\text{dec}}$ and every stored encoder state $\mathbf{h}_i^{\text{enc}}$ to quantify semantic relevance.
- SoftMax Weight Normalization:** Applies the SoftMax function over all raw similarity scores, converting them into normalized probability weights ($\alpha_{i,j} \in [0, 1]$) that sum to 1.0.
- Attention Context Vector Aggregation:** Computes a weighted sum of the encoder hidden states scaled by their corresponding attention weights, creating a dynamic, step-specific context vector $\mathbf{c}_j$.
- Output Projection Layer:** Concatenates the Attention Context Vector $\mathbf{c}_j$ with the current decoder hidden state $\mathbf{s}_j^{\text{dec}}$, feeding the combined representation through a Fully Connected (FC) layer and SoftMax to select the predicted output token.

3. Mathematical Formulations and Similarity Metrics

3.1 Similarity Metrics: Cosine Similarity vs. Dot Product

To determine how much influence an encoded input word i should have on decoding step j , the network evaluates the similarity between encoder hidden state $\mathbf{h}_i^{\text{enc}}$ and decoder state $\mathbf{s}_j^{\text{dec}}$.

- Cosine Similarity Formula:** Mathematically, Cosine Similarity calculates the normalized inner product of two numerical vectors:

$$\text{CosineSimilarity}(\mathbf{h}_i^{\text{enc}}, \mathbf{s}_j^{\text{dec}}) = \frac{\mathbf{h}_i^{\text{enc}} \cdot \mathbf{s}_j^{\text{dec}}}{\|\mathbf{h}_i^{\text{enc}}\| \|\mathbf{s}_j^{\text{dec}}\|} = \frac{\sum_{k=1}^D h_{i,k}^{\text{enc}} s_{j,k}^{\text{dec}}}{\sqrt{\sum_{k=1}^D (h_{i,k}^{\text{enc}})^2} \sqrt{\sum_{k=1}^D (s_{j,k}^{\text{dec}})^2}} \quad (1)$$

The denominator normalizes the score to fall strictly within $[-1.0, 1.0]$. This normalization is useful when comparing vectors of varying lengths or across different numbers of LSTM units.

2. Dot Product Similarity (Standard Approach): In practice, because the number of hidden units remains constant throughout the architecture, scaling by vector magnitude is unnecessary. The model uses the simplified **Dot Product** (the numerator of Cosine Similarity):

$$\text{Score}_{i,j} = \text{DotProduct}(\mathbf{h}_i^{\text{enc}}, \mathbf{s}_j^{\text{dec}}) = \mathbf{h}_i^{\text{enc}} \cdot \mathbf{s}_j^{\text{dec}} = \sum_{k=1}^D h_{i,k}^{\text{enc}} s_{j,k}^{\text{dec}} \quad (2)$$

Large positive dot products indicate high similarity, while negative values indicate opposite alignment.

Numerical Example: Dot Product Scoring

Suppose an Encoder processes the input phrase "let's go" with $D = 2$ hidden units:

- Encoder State for "let's": $\mathbf{h}_1^{\text{enc}} = [-0.76, 0.75]$
- Encoder State for "go": $\mathbf{h}_2^{\text{enc}} = [-0.30, 0.80]$
- Decoder State for <EOS> token: $\mathbf{s}_1^{\text{dec}} = [0.91, 0.38]$

Computing the dot products for decoding step 1:

$$\text{Score}_{1,1}(\text{"let's"}) = (-0.76 \times 0.91) + (0.75 \times 0.38) = -0.6916 + 0.2850 = -\mathbf{0.41}$$

$$\text{Score}_{2,1}(\text{"go"}) = (-0.30 \times 0.91) + (0.80 \times 0.38) = -0.2730 + 0.3040 = \mathbf{0.01}$$

Since $\text{Score}_{2,1}(0.01) > \text{Score}_{1,1}(-0.41)$, the word "go" exhibits higher alignment with the <EOS> decoding step than "let's".

3.2 SoftMax Attention Weight Normalization

Raw similarity scores are passed through the SoftMax function to convert them into positive attention weights $\alpha_{i,j}$ that sum to 1.0:

$$\alpha_{i,j} = \text{SoftMax}(\text{Score}_{i,j}) = \frac{e^{\text{Score}_{i,j}}}{\sum_{k=1}^N e^{\text{Score}_{k,j}}} \quad (3)$$

Using our numerical scores $[-0.41, 0.01]$:

$$\alpha_{1,1}(\text{"let's"}) = \frac{e^{-0.41}}{e^{-0.41} + e^{0.01}} \approx \frac{0.6636}{0.6636 + 1.0101} = \mathbf{0.40} \quad (40\%)$$

$$\alpha_{2,1}(\text{"go"}) = \frac{e^{0.01}}{e^{-0.41} + e^{0.01}} \approx \frac{1.0101}{0.6636 + 1.0101} = \mathbf{0.60} \quad (60\%)$$

These weights dictate that the decoder will utilize 40% of the "let's" encoding and 60% of the "go" encoding when generating the first output token.

3.3 Weighted Context Vector and Output Prediction

The Attention Context Vector $\mathbf{c}_j$ is formed by scaling each encoder hidden state by its attention weight and summing the results:

$$\mathbf{c}_j = \sum_{i=1}^N \alpha_{i,j} \mathbf{h}_i^{\text{enc}} = \alpha_{1,j} \mathbf{h}_1^{\text{enc}} + \alpha_{2,j} \mathbf{h}_2^{\text{enc}} + \dots + \alpha_{N,j} \mathbf{h}_N^{\text{enc}} \quad (4)$$

Finally, the context vector $\mathbf{c}_j$ and decoder state $\mathbf{s}_j^{\text{dec}}$ are concatenated and passed through a Fully Connected layer:

$$\hat{\mathbf{y}}_j = \text{SoftMax}(\mathbf{W}_{\text{out}}[\mathbf{c}_j; \mathbf{s}_j^{\text{dec}}] + \mathbf{b}_{\text{out}}) \quad (5)$$

Selecting the class with the highest SoftMax probability produces the translated token (e.g., "vamos").

4. Step-by-Step Operational Workflow

The operational execution of Attention during Seq2Seq decoding follows a structured 6-step loop:

1. **Encoding Phase:** Input sequence tokens $(x_1, x_2, \dots, x_N)$ pass through the Encoder LSTMs. All output hidden vectors $(\mathbf{h}_1^{\text{enc}}, \dots, \mathbf{h}_N^{\text{enc}})$ are preserved in memory.
2. **Decoder Initialization:** The initial context vector initializes the Decoder LSTMs. A start token ($\langle \text{EOS} \rangle$ or $\langle \text{SOS} \rangle$) is fed into the Decoder to generate the first decoder state $\mathbf{s}_1^{\text{dec}}$.
3. **Pairwise Score Calculation:** Compute dot products between $\mathbf{s}_1^{\text{dec}}$ and every stored encoder state $\mathbf{h}_i^{\text{enc}}$.
4. **Weight Normalization:** Pass the raw score vector through SoftMax to derive normalized attention percentage weights $\alpha_{i,1}$.
5. **Context Synthesis:** Multiply encoder vectors by their weights and sum them to construct the dynamic Attention Context Vector $\mathbf{c}_1$.
6. **Token Projection & Autoregressive Feedback:** Combine $\mathbf{c}_1$ and $\mathbf{s}_1^{\text{dec}}$ in the FC layer to predict token 1 (*"vamos"*). Feed *"vamos"* back as input for step 2, repeating steps 3–6 until $\langle \text{EOS} \rangle$ is emitted.

5. Summary Comparison: Standard Seq2Seq vs. Seq2Seq with Attention

Table 1: Architectural and Operational Comparison of Seq2Seq Models

Property / Aspect	Standard Seq2Seq (No Attention)	Seq2Seq with Attention
Encoder Output	Single compressed Context Vector	Full sequence of stored hidden states $\mathbf{h}_i^{\text{enc}}$
Decoder Access	Indirect (Only via initial state)	Direct access to all input step encodings
Context Representation	Static	Dynamic (Recalculated per step)
Alignment Mechanism	Implicit	Explicit via Dot Product & SoftMax weights
Long-Sentence Quality	Degrades severely (Information loss)	High fidelity (Preserves early tokens)
Computational Cost	$\mathcal{O}(N + M)$	$\mathcal{O}(N \times M)$ (Pairwise score matrices)

6. Conceptual Takeaways and Connection to Transformers

Attention fundamentally alters sequence modeling by eliminating the static memory bottleneck. By providing dynamic, direct connections between encoder states and decoder steps, it guarantees that critical tokens—regardless of sentence length—remain fully accessible. This mechanism serves as the direct evolutionary stepping stone to **Self-Attention** and **Transformer** architectures (such as GPT and BERT), which replace recurrent LSTMs entirely with parallelized attention pathways.



...and the second output from the Decoder is **<EOS>** so we are done decoding.

el vamos y <EOS>
0 0 0 1

SoftMax

-5 1.4 -2 7.4

-6 0.6 -8 0.9

0.78 0.22
1.26 -0.01

SoftMax

sum sum

-0.76 0.91

Encoder

